

Projet_Rust_F4

Simulation multi-threadee de robots collecteurs de ressources
ecrite en Rust, avec interface terminal Ratatui

Document de preparation a la soutenance
Concepts du projet, architecture, code explique, questions probables

Equipe (auteurs des commits) : iimAtomic, Gwenael Gueho, Voutsas Stevie, norab0, Lux
VEGBA
Prepared le 2 juillet 2026

Sommaire

1. Introduction et objectifs
2. Concept general de la simulation
3. Concepts Rust mobilises
4. Architecture generale (threads et etat partage)
5. Module par module : le code explique
6. La machine a etats du Collector
7. Synchronisation et securite memoire
8. Tests unitaires
9. Qualite du code (clippy, fmt) et corrections apportees
10. Historique et organisation du projet
11. Points forts a mettre en avant
12. Limites connues et pistes d'amelioration
13. Questions probables du jury et elements de reponse
14. Conclusion

1. Introduction et objectifs

Projet_Rust_F4 est un projet d'école réalisé en équipe (5 contributeurs identifiés dans l'historique Git : iimAtomic, Gwenael Gueho, Voutsas Stevie, norab0/Nora et Lux VEGBA), développé par parties successives (*partie 3, phase4, P1...*) puis fusionnées. L'objectif est de mettre en pratique, sur un cas concret, les fondamentaux du langage Rust les plus caractéristiques : gestion de la mémoire sans ramasse-miettes, concurrence multi-thread sûre, types algébriques (enum + pattern matching), et un peu d'algorithmique (parcours en largeur).

Le rendu est un **simulateur terminal** : une carte générée procéduralement, des robots autonomes qui l'explorent et en extraient des ressources, le tout affiché en temps réel avec la bibliothèque **Ratatui**.

Stack technique

Dependance	Rôle dans le projet
ratatui 0.29	Rendu de l'interface terminal (widgets, mise en page, couleurs)
crossterm 0.28	Accès bas niveau au terminal (mode raw, écran alternatif, clavier)
noise 0.9	Bruit de Perlin pour générer les obstacles de la carte
rand 0.8	Génération aléatoire (positions, quantités, choix de déplacement)
crossbeam-channel 0.5	Canal multi-producteurs / mono-consommateur entre robots et base

2. Concept general de la simulation

Une carte de **60 x 25** cases est generee au demarrage. Chaque case peut etre : vide, un obstacle, une ressource (**Energie** ou **Cristal**, entre 50 et 200 unites), ou la base (placee au centre de la carte).

Deux types de robots, deux comportements

- **Scout** : explore au hasard, case par case parmi ses voisines praticables. A chaque pas, il regarde ses 4 cases voisines et signale a la base tout ce qu'il decouvre (obstacle, ressource, case vide).
- **Collector** : ne connait que ce que la base a deja appris des scouts (fog of war). Il choisit la ressource connue la plus proche, calcule le chemin le plus court par un parcours en largeur (BFS), s'y rend, collecte petit a petit jusqu'a saturation de sa cargaison, puis rentre a la base pour la vider.

Le brouillard de guerre (fog of war)

La carte reelle (*Map*) et la carte connue de la base (*known_map*) sont deux structures distinctes. Les collecteurs ne lisent jamais la carte reelle pour decider ou aller : ils raisonnent uniquement sur *known_map*, ce qui simule une decouverte progressive du terrain, cellule affichee '?' tant qu'aucun robot ne l'a visitee.

Fin de la simulation

Des que *Map::has_resources()* renvoie faux (plus aucune case de type *Resource*), le drapeau d'arret est leve, tous les threads robots et le thread base se terminent proprement, et l'affichage bascule sur le message **Simulation terminee**.

3. Concepts Rust mobilises

Cette section liste, pour la soutenance, les notions Rust importantes illustrees par le projet -- avec un extrait de code a l'appui pour chacune.

3.1 Ownership, emprunt et types partages (Arc)

Chaque robot tourne sur son propre thread OS (`std::thread::spawn`). Pour qu'ils puissent tous lire/crire la meme carte sans copier les donnees, celle-ci est enveloppee dans un *Arc* (Atomic Reference Counted) : chaque thread recoit un clone de la reference (bon marche, incremente juste un compteur atomique), tous pointent vers la meme donnee en memoire.

```
pub type SharedMap = Arc<RwLock<Map>>;

let shared_map = Map::new_shared(MAP_WIDTH, MAP_HEIGHT);
let map = shared_map.clone(); // clone de l'Arc, pas de la Map
```

3.2 RwLock vs Mutex : choisir le bon verrou

La carte (*Map*) et la carte connue (*known_map*) sont proteges par un **RwLock** : de nombreux threads la lisent a chaque tick (scouts, collectors, boucle de rendu) mais peu l'ecrivent (un scout qui rapporte une decouverte, un collector qui collecte). Un **RwLock** autorise plusieurs lecteurs simultanes, ce qui evite de serialiser des lectures qui n'entrent pas en conflit entre elles.

L'etat d'interface (*UiState*) est protege par un simple **Mutex** : il est modifie frequemment en bloc par la base, et lu en bloc par le rendu -- pas de gain a distinguer lecture/ecriture pour une si petite structure.

3.3 Enums et pattern matching (types algebriques)

Le protocole de communication entre robots et base est un unique enum *RobotMessage*, dont chaque variante porte ses propres donnees. C'est l'exemple le plus parlant du systeme de types de Rust : impossible de traiter un message sans gerer explicitement chacune de ses variantes (le compilateur impose l'exhaustivite du *match*).

```
pub enum RobotMessage {
    ResourceFound { pos: (usize, usize), kind: ResourceKind, quantity: u32 },
    ObstacleFound { pos: (usize, usize) },
    CellSeen { pos: (usize, usize), cell: Cell },
    ResourceCollected { pos: (usize, usize), kind: ResourceKind, amount: u32 },
    RobotMoved { robot_id: usize, kind: RobotKind, pos: (usize, usize) },
}
```

De la meme maniere, l'etat d'un collecteur (*CollectorState*) est un enum a quatre variantes qui modelise une machine a etats finie -- voir section 6.

3.4 Canaux et passage de messages (crossbeam-channel)

Plutot que de faire partager aux robots un etat mutable commun (ce qui multiplierait les verrous et les risques de blocage), chaque robot envoie des messages a la base via un canal MPMC non borne. C'est une application directe de l'adage '*Do not communicate by sharing memory; share memory by communicating*' popularise par Go et repris dans l'ecosysteme Rust.

```
let (tx, rx) = unbounded::<messages::RobotMessage>();
let base = Base::new(rx, known_map.clone(), shared_ui.clone());
// chaque robot recoit un clone de tx et envoie ses decouvertes
let _ = self.sender.send(RobotMessage::ObstacleFound { pos: (nx, ny) });
```

3.5 Traits derives (Default, Debug, Clone, PartialEq)

Le projet utilise abondamment les macros de derivation pour eviter du code repetitif : `#[derive(Debug, Clone, Copy, PartialEq)]` sur `ResourceKind`, `#[derive(Default)]` sur `UiState` (tous les champs valent deja zero/faux/vide par default, inutile d'ecrire l'implementation a la main).

3.6 Gestion des erreurs et des verrous empoisonnes

`main` retourne un `io::Result<>` et propage les erreurs d'entree/sortie avec l'operateur `?`. Pour les verrous (`RwLock/Mutex`), le projet choisit deliberement de **recuperer** un verrou empoisonne (si un thread a panique en le tenant) plutot que de faire planter tout le programme :

```
let map = shared_map.read().unwrap_or_else(|e| e.into_inner());
```

Ce choix garantit que le rendu et les autres robots continuent de fonctionner meme si un thread isole a rencontre un probleme.

3.7 Securite face aux panics : `catch_unwind`

Si la simulation panique, le terminal doit imperativement etre restaure (sortir du mode raw et de l'ecran alternatif), sinon le terminal de l'utilisateur reste dans un etat casse apres la fermeture du programme. `main` encapsule donc l'execution dans `catch_unwind`, restaure le terminal dans tous les cas, puis repropage le panic s'il y en a eu un.

```
let result = catch_unwind(AssertUnwindSafe(|| run(&mut terminal)));
disable_raw_mode()?;
execute!(terminal.backend_mut(), LeaveAlternateScreen)?;
terminal.show_cursor()?;
match result {
    Ok(inner) => inner,
    Err(panic) => std::panic::resume_unwind(panic),
}
```

3.8 Let-chains (edition 2024)

Le projet cible l'edition 2024 de Rust, qui stabilise les *let-chains* : on peut combiner une condition booleenne et un `if let` dans le meme `if`, sans imbrication. Cela a permis de simplifier deux conditions du code (voir section 9).

```
if event::poll(Duration::from_millis(100))?
    && let Event::Key(_) = event::read()?
{
    break;
}
```

3.9 Iterateurs plutot que boucles indexees

Idiome Rust courant : parcourir une structure via ses iterateurs (`iter_mut().enumerate()`) plutot que par indices manuels, ce qui evite les acces hors-limites potentiels et se lit plus naturellement (voir section

9 pour le contexte clippy).

4. Architecture generale (threads et etat partage)

Le programme démarre **1 thread Base + N threads Scout + M threads Collector** (N = 2, M = 2 par défaut), plus le thread principal qui gère la boucle de rendu. Aucun robot ne modifie directement l'état d'un autre : tout transite par des messages vers la Base, qui est la seule propriétaire logique de *known_map* et de *UiState*.

```
Scout(s)  ----RobotMessage----> +-----+
                                     | Base |----> known_map (Arc>)
Collector(s) --RobotMessage----> | thread|----> UiState  (Arc>)
                                     +-----+

Map (Arc>) <---- lue/ecrite par scouts, collectors, et la boucle de rendu
```

Etat partage entre threads

Etat	Type	Role
shared_map	Arc<RwLock<Map>>	Carte réelle : obstacles, ressources et leurs quantités
known_map	Arc<RwLock<HashMap<(x,y), Cell>>>	Ce que la base sait déjà (fog of war)
shared_ui	Arc<Mutex<UiState>>	Compteurs et positions des robots pour l'affichage
stop	Arc<AtomicBool>	Signal d'arrêt partagé (fin de partie ou touche pressée)

Pourquoi un AtomicBool et pas un Mutex ?

Le drapeau d'arrêt est lu à haute fréquence par tous les threads robots (à chaque tick, 150 ms) et par la boucle de rendu. Un type atomique permet de le lire/écrire sans verrou, donc sans jamais bloquer -- adapté à une simple valeur booléenne partagée.

Arrêt propre de la simulation

Quand la boucle de rendu se termine (touche pressée ou plus de ressources), *main::run* lève *stop*, puis appelle *.join()* sur chaque handle de thread robot et sur le thread base, garantissant qu'aucun thread ne continue de tourner en arrière-plan après la fin du programme.

5. Module par module : le code explique

5.1 src/main.rs -- point d'entree

Configure le terminal (mode raw, ecran alternatif), lance tous les threads, execute la boucle de rendu, puis restaure le terminal. Contient aussi les constantes de configuration de la simulation.

Constante	Valeur	Effet
MAP_WIDTH / MAP_HEIGHT	60 / 25	Dimensions de la carte
SCOUT_COUNT	2	Nombre de robots eclaireurs
COLLECTOR_COUNT	2	Nombre de robots collecteurs
COLLECTOR_CAPACITY	5	Cargaison max avant retour a la base
ROBOT_TICK	150 ms	Delai entre deux actions d'un robot

5.2 src/map.rs -- generation et etat de la carte

Map::new genere la carte en 4 etapes : (1) placer des obstacles a partir d'un bruit de Perlin, sauf autour de la base (rayon de degagement) ; (2) tirer aleatoirement des positions libres et y placer des ressources Energie/Cristal jusqu'a atteindre la densite voulue (6% des cases) ; (3) garantir qu'il existe toujours au moins une ressource, meme si le tirage aleatoire a echoue, via un filet de securite *find_first_empty* ; (4) placer la base.

```
let n = perlin.get([x as f64 * OBSTACLE_SCALE, y as f64 * OBSTACLE_SCALE]);
if n > OBSTACLE_THRESHOLD {
    *cell = Cell::Obstacle;
}
...
if placed == 0 && width > 0 && height > 0
    && let Some((x, y)) = find_first_empty(&cells, base_pos)
{
    cells[y][x] = Cell::Resource(Resource { kind: ResourceKind::Energy, .. });
}
```

Map::try_collect encapsule la logique de prelevement : elle retire au plus *amount* unites d'une ressource, retourne son type et la quantite restante, et transforme la case en *Cell::Empty* quand elle est epuisee.

5.3 src/messages.rs -- protocole de communication

Definit le vocabulaire echange entre robots et base :

Message	Emis par	Signification
ResourceFound	Scout	Une ressource a ete decouverte a une position
ObstacleFound	Scout	Un obstacle a ete decouvert a une position
CellSeen	Scout	Une case vide ou la base a ete vue (complete le fog of war)
ResourceCollected	Collector	Une quantite de ressource a ete rapportee

Message	Emis par	Signification
RobotMoved	Scout / Collector	La position courante du robot a change (pour l'affichage)

5.4 src/base.rs -- le thread central

La *Base* tourne en boucle sur *receiver.recv_timeout(50 ms)* : elle traite chaque message des qu'il arrive, sinon elle verifie periodiquement si le signal d'arret a ete leve. A chaque message, elle met a jour *known_map* et/ou les compteurs *energy_collected* / *crystals_collected*, puis synchronise *UiState*.

```
match self.receiver.recv_timeout(Duration::from_millis(50)) {
    Ok(msg) => { self.handle(msg); self.process_messages(); }
    Err(RecvTimeoutError::Timeout) => {
        if stop.load(Ordering::Relaxed) { self.process_messages(); break; }
    }
    Err(RecvTimeoutError::Disconnected) => { self.process_messages(); break; }
}
```

Ce pattern gere trois cas : un message est arrive (le traiter), rien n'est arrive dans le delai (verifier si on doit s'arreter), ou l'emetteur a ferme le canal (tous les robots sont partis : s'arreter).

5.5 src/robot.rs -- intelligence des robots

Scout

A chaque tick : regarde ses 4 voisins non deja signales, envoie le message adapte (obstacle / ressource / case vue), puis choisit au hasard une case voisine praticable pour s'y deplacer (*rand::seq::SliceRandom::choose*).

Collector : selection de cible et BFS

select_target parcourt *known_map* et retourne la ressource connue non epuisee la plus proche (distance de Manhattan). *bfs_path* calcule ensuite le chemin le plus court sur la grille en 4-connexite avec un parcours en largeur classique (file, tableau de cases visitees, table des parents pour reconstruire le chemin) :

```
let mut visited = vec![vec![false; map_width]; map_height];
let mut parent = HashMap::new();
let mut queue = VecDeque::new();
queue.push_back(start);
while let Some((x, y)) = queue.pop_front() {
    for (nx, ny) in neighbors(x, y, map_width, map_height) {
        if visited[ny][nx]
            || matches!(known_map.get(&(nx, ny)), Some(Cell::Obstacle)) {
            continue;
        }
        visited[ny][nx] = true;
        parent.insert((nx, ny), (x, y));
        if (nx, ny) == goal { return Some(reconstruct_path(&parent, start, goal)); }
        queue.push_back((nx, ny));
    }
}
```

Le BFS ne connait que *known_map* (jamais la carte reelle) : un obstacle non encore decouvert par un scout est donc traverse par erreur dans le calcul de chemin -- c'est une consequence assumee du fog of war, pas un bug.

5.6 src/ui.rs -- rendu Ratatui

render decoupe l'ecran en deux zones horizontales : la carte (largeur flexible) et un panneau lateral fixe de 24 colonnes (stats + legende). *render_map* dessine d'abord le symbole de chaque case (via *cell_symbol*), puis superpose les robots presents sur cette case (glyphe x rouge pour un scout, o magenta pour un collecteur -- les robots masquent donc le sol qu'ils occupent).

```
fn cell_symbol(pos, base_pos, known_cell: Option<&Cell>) -> (char, Color) {
    if pos == base_pos { return ('#', Color::LightGreen); }
    match known_cell {
        Some(Cell::Empty) => ('.', Color::DarkGray),
        Some(Cell::Obstacle) => ('O', Color::LightCyan),
        Some(Cell::Resource(r)) => match r.kind { ... },
        Some(Cell::Base) => ('#', Color::LightGreen),
        None => ('?', Color::DarkGray),
    }
}
```

6. La machine a etats du Collector

L'enum *CollectorState* modelise le cycle de vie d'un collecteur en quatre etats. A chaque tick, *Collector::step* consomme l'etat courant et produit le suivant.

```
Idle
-> si cargo plein          : ReturningToBase(chemin vers la base)
-> sinon si ressource connue : MovingToResource(chemin BFS)
-> sinon                   : Idle (attend une decouverte des scouts)

MovingToResource(chemin)
-> chemin non vide : avance d'une case, notifie RobotMoved
-> chemin vide     : Collecting si une ressource est bien presente, sinon Idle

Collecting { pos }
-> preleve 1 unite (Map::try_collect), envoie ResourceCollected
-> cargo plein ou ressource epuisee : ReturningToBase
-> sinon                           : reste en Collecting

ReturningToBase(chemin)
-> chemin non vide : avance d'une case, notifie RobotMoved
-> chemin vide     : vide la cargaison (cargo = 0), retourne a Idle
```

Ce decoupage en etats explicites (plutot qu'un ensemble de booleens) rend chaque transition exhaustive et verifiee par le compilateur : impossible d'oublier de gerer un cas, le *match* ne compile pas si une variante manque.

7. Synchronisation et securite memoire

7.1 Verrous courts, releases explicites

Dans `Collector::step` (etat `Collecting`), le verrou d'ecriture sur la carte est explicitement libere (`drop(map)`) avant d'envoyer un message sur le canal ou de modifier `self.cargo`. Cela evite de garder un verrou plus longtemps que necessaire et supprime tout risque qu'un envoi de message bloque pendant qu'un verrou est tenu.

```
let mut map = map.write().unwrap_or_else(|e| e.into_inner());
match map.try_collect(pos, 1) {
    Some((kind, taken, remaining)) => {
        drop(map); // le verrou est relache avant d'envoyer le message
        self.cargo += taken;
        let _ = self.sender.send(RobotMessage::ResourceCollected { pos, kind, amount: taken });
        ...
    }
}
```

7.2 Instantanes (snapshots) pour le rendu

A chaque frame, la boucle de rendu clone `known_map` (`.clone()` sur le `HashMap`) avant de dessiner, plutot que de garder le verrou de lecture pendant tout le dessin. Cela minimise la duree de detention du verrou et evite de bloquer les robots pendant le rendu, au prix d'une copie ponctuelle (acceptable vu la taille de la carte).

7.3 Recuperation des verrous empoisonnes

Rust empoisonne un `Mutex/RwLock` si le thread qui le tenait a paniqué, par securite. Le projet choisit systematiquement `unwrap_or_else(|e| e.into_inner())` pour recuperer les donnees malgre tout, plutot que de propager le panic a tous les threads consommateurs -- un choix assume pour la resilience de la simulation.

8. Tests unitaires

7 tests, écrits en modules `#[cfg(test)]` a cote du code qu'ils verifient :

Test	Ce qu'il verifie
<code>map::new_places_base_and_correct_dimensions</code>	Dimensions de la carte et presence de la base a la bonne position
<code>map::generated_resources_use_required_quantities</code>	Les quantites generees restent entre 50 et 200
<code>map::generated_base_area_stays_passable</code>	Aucun obstacle dans le rayon de degagement autour de la base
<code>map::try_collect_depletes_and_clears_cell</code>	Le prelevement diminue la quantite et vide la case a zero
<code>ui::unknown_cell_uses_question_mark</code>	Une case jamais vue s'affiche '?'
<code>ui::base_is_visible_even_if_unknown</code>	La base reste visible meme sans decouverte prealable
<code>ui::known_crystal_uses_required_symbol_and_color</code>	Le symbole/couleur d'un cristal connu est correct

Commande : `cargo test -- resultat actuel : 7 passed; 0 failed.`

9. Qualite du code (clippy, fmt) et corrections apportees

Avant la soutenance, le projet a ete verifie avec *cargo check*, *cargo build*, *cargo test*, *cargo fmt --check* et *cargo clippy --all-targets*. **Aucune erreur de compilation** n'a jamais existe. Clippy signalait en revanche 5 avertissements de style, tous corriges :

Fichier	Avertissement clippy	Correction appliquee
map.rs:49-50	needless_range_loop (boucle indexee inutile)	Remplacement de for y in 0..height / for x in 0..width par cells.iter_mut().e
map.rs:90-97	collapsible_if (if imbrique inutilement)	Fusion en une seule condition grace aux let-chains (if cond && let Some(
ui.rs:23-32	derivable_impls (impl Default manuel evitable)	Remplacement de l'impl Default manuel par #[derive(Default)] (tous les c
main.rs:140-144	collapsible_if (if imbrique inutilement)	Fusion avec un let-chain : if event::poll(...)? && let Event::Key(_) = event:

Etat final : **0 erreur, 0 avertissement clippy, formatage conforme a rustfmt, 7 tests au vert.**

10. Historique et organisation du projet

Le depot est un petit projet collaboratif construit en une journee de developpement intense (2026-07-01), avec une progression par 'parties' assignees a differentes personnes, fusionnees via pull requests :

```
6fdfc1a init
c611938 partie 3 (gwenael9)
a44eaa5 Merge PR #1 branche/gwenael (Gwenael Gueho)
ec8c82c Merge PR #2 (Lux VEGBA)
986099b phase4 (iimAtomic)
102a650 Merge PR #3 branch/lux (Lux VEGBA)
892e52c clean code; refactor (Voutsas Stevie)
a044ace add p1 map and tui (norab0)
48fcbe1 Merge branch/nora (conflits resolus sur tous les fichiers)
8dc19ee ameliore brouillard (norab0)
5a44dfe teste affichage (norab0)
1b2272f ajoute compteur (norab0)
37ec8f3 ajoute doc (norab0)
```

Le README initial ne decrivait que la 'Partie P1' (carte + TUI) ; il a ete entierement reecrit pour ce document et pour le depot afin de refleter l'etat final du projet (robots, messagerie, compteurs).

11. Points forts a mettre en avant

- **Architecture par passage de messages** plutot que par etat partage mutable entre robots : plus facile a raisonner, moins de risques de blocage mutuel (deadlock).
- **Fog of war realiste** : separation stricte entre la carte reelle et la carte connue, qui conditionne vraiment le comportement des collecteurs (pas juste un effet visuel).
- **Algorithme classique bien applique** : parcours en largeur (BFS) pour le plus court chemin sur une grille non ponderee.
- **Robustesse** : restauration garantie du terminal meme en cas de panic (catch_unwind), recuperation des verrous empoisonnes plutot que crash en cascade.
- **Qualite d'outillage** : projet formate (rustfmt), sans avertissement clippy, avec une suite de tests unitaires couvrant la generation de carte et le rendu.
- **Utilisation idiomatique de Rust recent** : edition 2024, let-chains, iterateurs plutot que boucles indexees, derive(Default).

12. Limites connues et pistes d'amélioration

- **Pas de reservation de ressource** : rien n'empêche deux collecteurs de *select_target* vers la même case si elle est la plus proche pour tous les deux. Ce n'est pas un bug (*try_collect* gère la quantité restante correctement), mais une inefficacité possible -- un mécanisme de reservation par ressource serait une amélioration naturelle.
- **Chemin BFS sur connaissance partielle** : un obstacle non encore découvert n'est pas évité par le BFS puisqu'il ne figure pas dans *known_map* ; le collecteur peut donc tenter de traverser une case qui se révèle bloquée (à gérer par un recalcul de chemin).
- **Configuration figée** : taille de carte et nombre de robots sont des constantes de compilation, pas des paramètres de ligne de commande.
- **Pas de graine (seed) loggée** : chaque exécution est différente, ce qui rend une partie précise impossible à rejouer pour du débogage.
- **Tests unitaires uniquement** : la logique multi-thread (ordonnancement réel des messages, conditions de concurrence) n'est vérifiée par aucun test d'intégration dédié.

13. Questions probables du jury et elements de reponse

Q. Pourquoi utiliser un RwLock pour la carte plutot qu'un simple Mutex ?

R. Parce que la carte est lue tres frequemment par plusieurs threads en parallele (scouts, collecteurs, boucle de rendu) et ecrite rarement. Un RwLock autorise plusieurs lecteurs simultanés, un Mutex forcerait a serialiser meme les lectures qui ne se genent pas entre elles.

Q. Que se passe-t-il si un thread robot panique ?

R. Le verrou qu'il tenait eventuellement devient 'empoisonne', mais le code recupere systematiquement les verrous empoisonnes (`unwrap_or_else(|e| e.into_inner())`) : les autres threads continuent de fonctionner. Si c'est le thread principal (la boucle de rendu) qui panique, `catch_unwind` dans `main()` garantit que le terminal est quand meme restaure avant que le programme ne se termine.

Q. Pourquoi les collecteurs ne consultent-ils jamais la carte reelle pour se deplacer ?

R. Pour simuler le brouillard de guerre : un collecteur ne doit connaitre que ce que les scouts (ou lui-meme) ont deja rapporte a la base. C'est `known_map`, alimentee uniquement par les messages recus, qui sert de source de verite pour la navigation.

Q. Comment le BFS garantit-il le chemin le plus court ?

R. Sur une grille non ponderee en 4-connexite, un parcours en largeur visite les cases par distance croissante depuis le depart (couche par couche). La premiere fois que la case cible est atteinte, c'est necessairement par un chemin de longueur minimale.

Q. Pourquoi crossbeam-channel plutot que std::sync::mpsc ?

R. `crossbeam-channel` offre une implementation MPMC performante et une API tres proche du `mpsc` standard (mêmes methodes `send/recv`), avec de meilleures performances et un canal non borne pratique ici puisque le volume de messages depend du nombre de robots et de la vitesse de decouverte, difficile a borner a priori.

Q. Que se passe-t-il si deux robots decouvrent la meme case en meme temps ?

R. Chacun envoie son propre message a la base ; la base les traite l'un apres l'autre (elle est mono-thread, `receiver.recv_timeout` en boucle), donc pas de course critique : le dernier message traite pour une position donnee ecrase simplement la valeur precedente dans `known_map`, ce qui est sans consequence ici car les deux messages decrivent la meme realite.

Q. Comment est garantie la coherence entre l'affichage et l'etat reel pendant qu'il change en continu ?

R. Chaque frame prend un instantane coherent : elle lit la carte, clone `known_map` et lit `UiState` au meme instant, puis dessine a partir de ces copies. L'image peut etre legerement en retard sur l'etat courant (quelques millisecondes) mais jamais incoherente en elle-meme.

Q. Pourquoi la machine a etats du collecteur est-elle un enum et pas des booleans ?

R. Parce qu'un enum avec pattern matching exhaustif empeche par construction les etats invalides (par exemple 'en train de collecter ET de rentrer a la base' n'existe pas) et le compilateur refuse de compiler si un cas du match n'est pas traite -- contraire a une combinaison de booleans qui autoriserait des etats incoherents.

Q. Le projet est-il teste ?

R. Oui, par 7 tests unitaires sur la generation de carte (dimensions, quantites de ressources, zone de la base) et sur le rendu (symboles/couleurs). Les aspects multi-thread (ordonnancement reel) ne sont pas couverts par des tests automatises, c'est une limite assumee du projet.

14. Conclusion

Projet_Rust_F4 illustre, sur un exemple concret et visuel, les points forts de Rust pour la programmation concurrente : partage de données sur plusieurs threads sans ramasse-miettes (Arc), synchronisation explicite et choisie à bon escient (RwLock vs Mutex vs Atomic), architecture par passage de messages plutôt que par état partagé, et un système de types (enums + pattern matching exhaustif) qui rend les états invalides irrepresentables. Le code est aujourd'hui sans erreur ni avertissement, teste, formate, et documente -- une base saine pour continuer à l'améliorer (reservation de ressources, configuration paramétrable, tests d'intégration multi-thread).

Bonne chance pour la soutenance.